Hoja 4: Ejercicio 19

TEMA

Diego Rodríguez Cubero

UCM

1 de diciembre de 2025

Contenidos

1. Enunciado del problema
2. Idea del algoritmo
3. Algoritmo
4. Conclusión

Enunciado del problema

Ejercicio 19

Nos dan un grafo $G = <V, A>$ no dirigido, valorado y conexo y un ARM (árbol de recubrimiento de coste mínimo) para él. También nos dan una arista nueva $(u, v) \notin A$, con un valor asociado c. Se pide un algoritmo de coste en $O(|V|)$ que modifique el ARM de manera que el resultado sea un ARM para $G' = <V, A \cup \{(u, v)\}>$. Razona la corrección y el coste.

Idea del algoritmo

- Al añadir la arista $(u, v) = e$ al ARM dado, se forma un ciclo.
- Para volver a tener un ARM, debemos eliminar una arista del ciclo formado, la de mayor coste.
- Tendremos que recorrer el ciclo para encontrar dicha arista. Esto puede hacerse encontrando el camino entre u y v en el ARM, y comparando su arista más cara con e.
- Si e es menor que la arista más cara del ciclo, eliminamos esa arista del ARM y añadimos e en su lugar.
- Si no, el ARM permanece igual.

Algoritmo

Listing 1: Actualización del ARM al añadir una arista

```
function actualizarARM(ARM, u, v, c):
// e = arista(u, v) con coste c
camino = encontrarCamino(ARM, u, v) // DFS o BFS , O(V)
aristaMax = arista de mayor coste en camino // O(V)
if (c < coste(aristaMax)):
    eliminar aristaMax de ARM // O(1)
    añadir e a ARM // O(1)
return ARM
```

Análisis de coste

- Encontrar el camino entre u y v en el ARM puede hacerse con DFS o BFS, ambos con coste $O(V + E)$, y en un árbol $E = V - 1$, por lo que es $O(V)$.
- Encontrar la arista de mayor coste en el camino también cuesta $O(V)$, al tener que recorrer el camino de V nodos en el peor caso.
- Las operaciones de eliminar y añadir aristas al ARM son $O(1)$.
- Por lo tanto, el coste total del algoritmo es $O(V)$.

Corrección del algoritmo

- Al añadir la arista (u, v) al ARM, se forma un ciclo.
- Aplicando la propiedad del corte, podemos cortar al ciclo formado en $G' = <V, A \cup \{(u, v)\}> = G + e$, cortanto en e y la arista de mayor coste del ciclo e_o, garantizamos que el árbol resultante sigue siendo un ARM.
- La propiedad del corte nos dice que exactamente una de las aristas e, e_o debe pertenecer al ARM óptimo.
- Si $c < coste(e_o)$, entonces e debe estar en el ARM óptimo, y eliminamos e_o.
- Si $c \geq coste(e_o)$, entonces e_o debe estar en el ARM óptimo, y no hacemos ningún cambio.
- Por lo tanto, el algoritmo es correcto.

Conclusión

- Hemos diseñado un algoritmo eficiente para actualizar un ARM tras la adición de una nueva arista.
- El algoritmo tiene un coste de $O(V)$, cumpliendo con los requisitos del problema.
- La corrección del algoritmo se debe a la aplicación de la propiedad del corte en grafos.